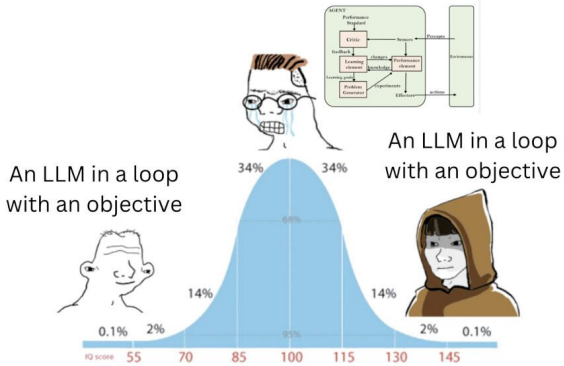


THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

How Agents Act: Tools and Frameworks

We want to build agents!

- LLMs produce text, we want to be able to do things
- Two definitions of “agent”
  - An entity that can observe and change their environment to accomplish a goal
  - LLM in a loop with tools
- *How can these be equivalent?*



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Reasoning is insufficient

Correct Reasoning:

Question: John takes care of 10 dogs. Each takes 0.5 hours a day to walk. How many hours per week does John spend walking dogs?

Thought: … That is 10 dogs * 0.5 hours / day or 5 hours per day. There are 7 days a week, so 5 hours / day * 7 = 35 hours a week.

Result: Correct

Failed Reasoning:

Question: How many molecules of water are in Lake Michigan?

Thought: … Lake Michigan is 2000 cubic miles. There are 4.168 * 10^12 liters per cubic mile. So there are 6.3 * 10^ 10 liters of water in Lake Michigan …

Result: Wrong

What are tools?

A mechanism for LLMs to access external information during reasoning

- Questions:
  - How to get an LLM to use tools?
  - How do LLM outputs turn into external requests?

Question: How many molecules of water are in Lake Michigan?

Search: “Volume of Lake Michigan”
Calculator: 4.168 * 10^12 x …

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Getting a model to use tools

Prompting to the rescue!

ReAct Loop:

- **[‘Thought’]:** <Reasoning here>
- **[‘Action’]:** <Tool call here>

What is the elevation range for the area that the eastern sector of the Colorado orogeny extends into

Thought: I need to search Colorado orogeny, find the area that the eastern sector of the Colorado orogeny extends into, then find the elevation range of the area.

Action: Search[Colorado orogeny]

Observation: The Colorado orogeny was an episode of mountain building (an orogeny) in Colorado and surrounding areas.

We need to train models to call tools

- Tool-Calls push a model out-of-distribution
- Self-supervised dataset creation
  - Given a context, suggest API calls
  - Insert <call, result> into text per suggestion
  - Perform prediction and select
  - Train on new dataset

Tool calling is now embedded into almost all modern models!

Bridging between models and tools

The LLM expresses itself in language: e.g., "*please run a program to compute X*" or "*compute the negative log of 0.01*"

Meanwhile:

- A "tool" might be an executable, Python function, workflow, …
- It might expect input, and produce output, in a variety of formats
- It might run on your PC, on an HPC system, in the cloud
- It might require specialized permissions to use
- The request may refer (implicitly or explicitly) to past interactions
- Available tools may change over time or depend on user identity

Activity 3: Native Tool Calling

- Think of a tool that would be useful for an agent to use (e.g., a calculator, MD simulation, protein docking)
- Prompt a model with the instruction to use this tool
- Review the response that is returned from the OpenAPI interface
- Parse that response to turn it into a tool invocation

Activity Analysis: Native Tool Calling

- What goes wrong?
- What happens when the model responds with a tool call that doesn't exist?
- What happens when we can't parse the call correctly?
- What happens when the result is not text?
- Or the result is too much text?

Structured outputs

- Problem:
  - Text generated by LLMs is sensitive to prompt formatting and phrasing
  - downstream code needs parseable output

- **Schema**: a formal description of the expected fields and types
  - Adds reliability, easy validation, and fewer retries

Structured outputs: JSON mode

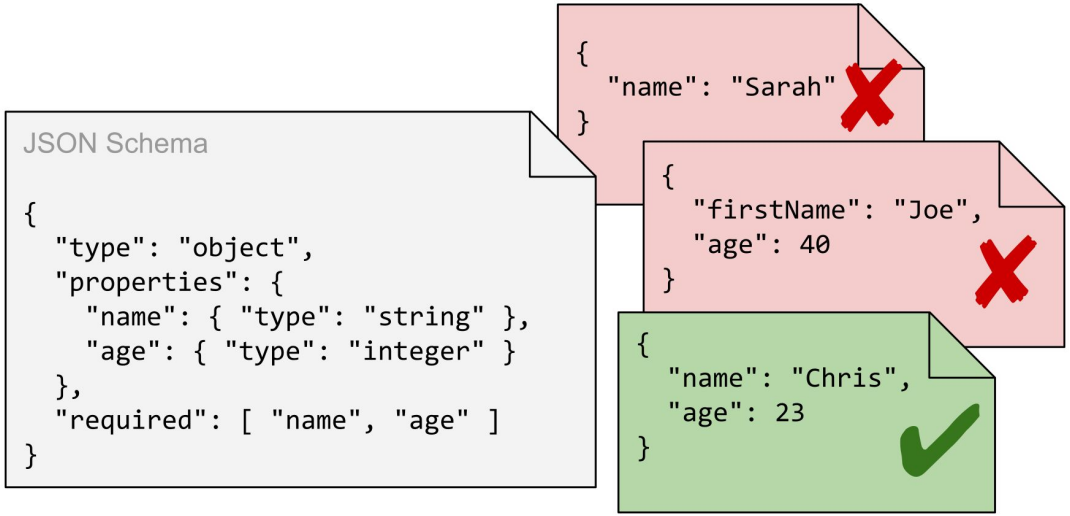


Image: example of JSON schema. https://jsoneditoronline.org/_astro/json-schema-validator.1GIn_IxH_QMpMA.svg

- **JSON mode:** API setting that forces the output of a language model to be a syntactically valid JSON schema rather than plain text

Structured outputs with Pydantic

- Define schemas as Python classes
- Pydantic gives type validation and auto-generates JSON Schema

```python
from pydantic import BaseModel

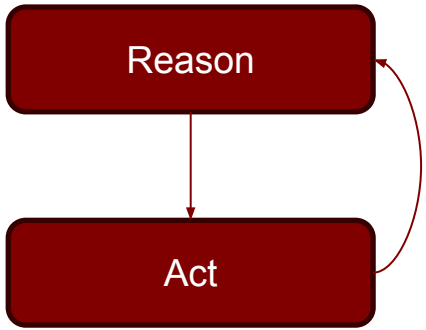
class Solution(BaseModel):
    reasoning: str    # step-by-step work first
    answer: float     # final answer last
```

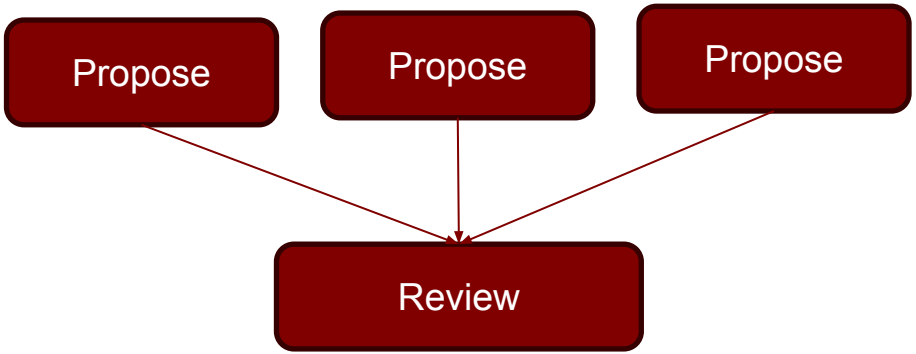
Pydantic: using the schema

- Schema sent to the API as:
  - response-format constraint
  - tool definition the model must call
- Validation errors can be fed back to the model for automatic retry

PydanticAI, LangGraph and Friends!

A graph is a general representation of prompting patterns and tool calling





THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

PydanticAI and LangGraph



- Structured outputs. LLM responses are validated against Pydantic models, with automatic retries when validation fails.
- Type safety. Type hints catch errors in your editor and at runtime.
- Simple tool calling. Plain Python functions become tools the agent can use.
- Dependency injection. It passes context, such as database connections and config, cleanly into agents and tools.

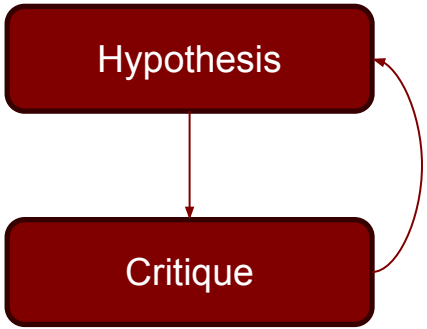


- Graph-based orchestration. You define agents as nodes and edges with shared state, including loops and branching.
- Persistence. Checkpointing lets long-running workflows pause, resume, and recover from failures.
- Human-in-the-loop. Workflows can stop for review or approval at any step.
- Multi-agent support. It coordinates several agents within a single workflow.

Activity 4: Agentic frameworks

Use LangGraph and Pydantic AI to build a structured LLM-workflow

1. Propose a hypothesis using the context of an abstract or paper
2. Critiques that hypothesis
3. Updates the hypothesis based on the critique



Questions

- How do you see the review scores for the hypothesis improving over iterations?
- Where do you think a more intelligent model could improve the hypothesis the most?
- What tools would help in this process?